

OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Note jeremy Est ce qu'il faut faire le calcul de la sous nutrition sur les pays qu'on a ? Est ce qu'il faut faire des graphiques ? Rajouter le soja La liste des céréales est difficile a trouver ...

1.1 - Importation des librairies

```
In [7]: #Importation de la librairie Pandas  
import pandas as pd
```

1.2 - Chargement des fichiers Excel

```
In [8]: #Importation du fichier population.csv
population = pd.read_csv('Projet 4/population.csv')

#Importation du fichier dispo_alimentaire.csv
Dispo_alimentaire = pd.read_csv('Projet 4/dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
Aide_alimentaire = pd.read_csv('Projet 4/aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
Sous_nutrition = pd.read_csv('Projet 4/sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

```
In [9]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(population.shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

Le tableau comporte 1416 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)

```
In [10]: #Consulter le nombre de colonnes
display(population.columns)
#La nature des données dans chacune des colonnes
display(population.dtypes)
#Le nombre de valeurs présentes dans chacune des colonnes
Valeurs_colonnes = population.count()
display(Valeurs_colonnes)
```

```
Index(['Zone', 'Année', 'Valeur'], dtype='object')
Zone      object
Année     int64
Valeur    float64
dtype: object
Zone      1416
Année     1416
Valeur    1416
dtype: int64
```

```
In [11]: #Affichage les 5 premières lignes de la table
population.head(5)
```

```
Out[11]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
In [12]: #Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier
#Multiplication de la colonne valeur par 1000
population['Valeur'] = population['Valeur']*1000
population
```

```
Out[12]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0
...	...	...	...
1411	Zimbabwe	2014	13586707.0
1412	Zimbabwe	2015	13814629.0
1413	Zimbabwe	2016	14030331.0
1414	Zimbabwe	2017	14236595.0
1415	Zimbabwe	2018	14438802.0

1416 rows × 3 columns

```
In [13]: #changement du nom de la colonne Valeur par Population
population.rename(columns={'Valeur':'Population'},inplace=True)
```

```
In [14]: #Affichage les 5 premières lignes de la table pour voir les modifications
population.head(5)
```

```
Out[14]:
```

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
In [15]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(Dispo_alim
```

comporte 15605 observation(s) ou article(s)

```
In [16]: #Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(Dispo_alimentaire.shape[1]))
```

Le tableau comporte 18 colonne(s)

```
In [17]: #Affichage les 5 premières lignes de la table
Dispo_alimentaire.head(5)
```

```
Out[17]:
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0

```
In [18]: #remplacement des NaN dans le dataset par des 0
Dispo_alimentaire.fillna(0,inplace=True)
Dispo_alimentaire.head()
```

```
Out[18]:
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0

```
In [19]: #multiplication de toutes les lignes contenant des milliers de tonnes en Kg
Colonnes_a_convertir = ["Aliments pour animaux","Autres Utilisations","Dispo"]
for Colonne in Colonnes_a_convertir:
    Dispo_alimentaire[Colonne]=Dispo_alimentaire[Colonne]*1000
```

```
In [20]: #Affichage les 5 premières lignes de la table
Dispo_alimentaire.head()
```

Out[20]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0

2.3 - Analyse exploratoire du fichier aide alimentaire

```
In [21]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(Aide_alim
```

Le tableau comporte 1475 observation(s) ou article(s)

```
In [22]: #Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(Aide_alimentaire.shape[1]))
```

Le tableau comporte 4 colonne(s)

```
In [23]: #Affichage les 5 premières lignes de la table
Aide_alimentaire.head()
```

Out[23]:

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
In [24]: #changement du nom de la colonne Pays bénéficiaire par Zone
Aide_alimentaire.rename(columns={'Pays bénéficiaire': 'Zone'}, inplace=True)
```

```
In [25]: #Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000
Aide_alimentaire['Valeur'] = Aide_alimentaire['Valeur'] * 1000
```

```
In [26]: #Affichage les 5 premières lignes de la table
Aide_alimentaire.head()
```

Out[26]:

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

```
In [27]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(Sous_nutr
Le tableau comporte 1218 observation(s) ou article(s)
```

```
In [28]: #Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(Sous_nutrition.shape[1]))
Le tableau comporte 3 colonne(s)
```

```
In [29]: #Afficher les 5 premières lignes de la table
Sous_nutrition.head()
```

Out[29]:

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
In [30]: #Conversion de la colonne sous nutrition en numérique
Sous_nutrition['Valeur'] = pd.to_numeric(Sous_nutrition["Valeur"],errors='coerce')
Sous_nutrition['Valeur'] = Sous_nutrition['Valeur'].fillna(0)
```

Conversion de la colonne (avec l'argument `errors=coerce` qui permet de convertir automatiquement les lignes qui ne sont pas des nombres en NaN)

Puis remplacement des NaN en 0

```
In [31]: #changement du nom de la colonne Valeur par sous_nutrition
Sous_nutrition.rename(columns={'Valeur':'sous_nutrition'},inplace=True)
```

```
In [32]: #Multiplication de la colonne sous_nutrition par 1000000
Sous_nutrition['sous_nutrition']=Sous_nutrition['sous_nutrition']*1000000
```

```
In [33]: #Afficher les 5 premières lignes de la table
Sous_nutrition.head()
```

```
Out[33]:
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

```
In [34]: # Il faut tout d'abord faire une jointure entre la table population et la table sous_nutrition
# filtrer sur l'année 2017 puis lier la jointure
Pop_2017 = population.loc[population['Année'] == 2017]
Pop_2017 = Pop_2017.drop(columns = 'Année')
SS_nut_2017 = Sous_nutrition.loc[Sous_nutrition['Année'] == '2016-2018']
SS_nut_2017 = SS_nut_2017.drop(columns = 'Année')
Data_nutrition_2017 = pd.merge(Pop_2017,SS_nut_2017,on = 'Zone')
display(Data_nutrition_2017)
```

	Zone	Population	sous_nutrition
0	Afghanistan	36296113.0	10500000.0
1	Afrique du Sud	57009756.0	3100000.0
2	Albanie	2884169.0	100000.0
3	Algérie	41389189.0	1300000.0
4	Allemagne	82658409.0	0.0
...	...	...	...
198	Venezuela (République bolivarienne du)	29402484.0	8000000.0
199	Viet Nam	94600648.0	6500000.0
200	Yémen	27834819.0	0.0
201	Zambie	16853599.0	0.0
202	Zimbabwe	14236595.0	0.0

203 rows × 3 columns

```
In [35]: #Affichage du dataset
Data_nutrition_2017.shape
```

```
Out[35]: (203, 3)
```

```
In [36]: #Calcul et affichage du nombre de personnes en état de sous nutrition
Nb_pers_ss_nut = Data_nutrition_2017['sous_nutrition'].sum()
print (Nb_pers_ss_nut)
```

```
535700000.0
```

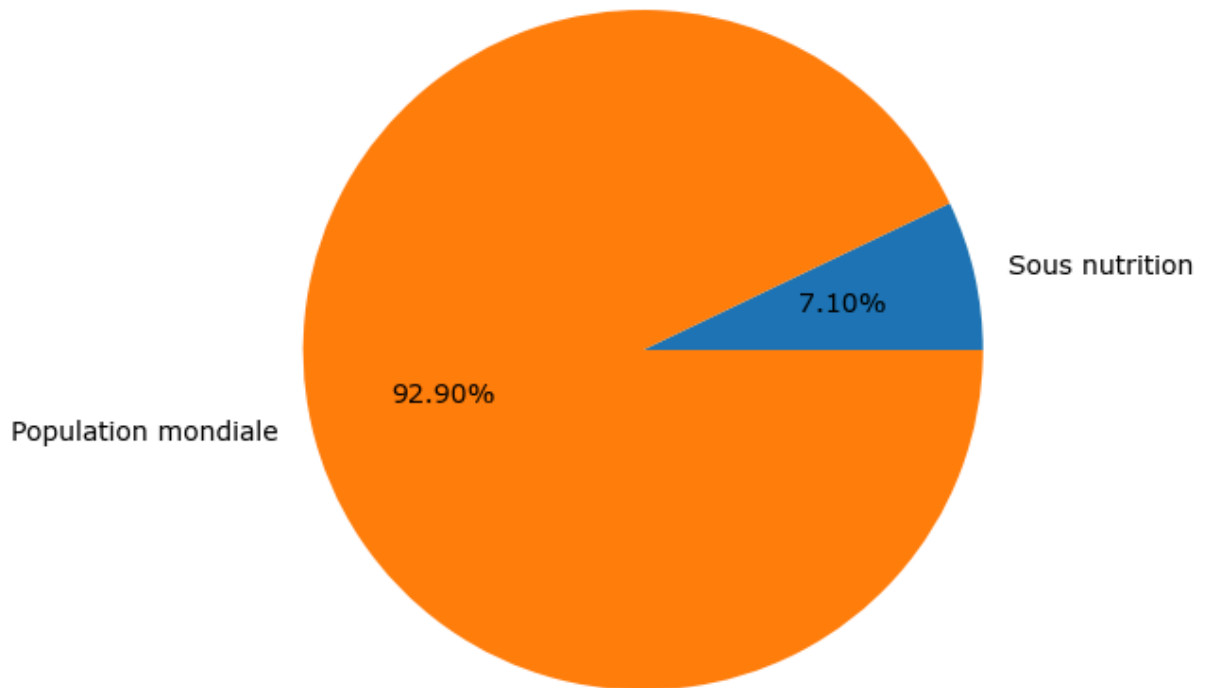
```
In [37]: # prendre le nombre de personnes en sous nutr, 2 colonnes résultats => % tot
Pop_total = Data_nutrition_2017['Population'].sum()
print (Pop_total)
Ratio_sous_nut_total =(Nb_pers_ss_nut/Pop_total)*100
print(Ratio_sous_nut_total)
```

```
7543798779.0
```

```
7.1011968332354165
```

```
In [38]: import matplotlib.pyplot as plt
import seaborn as sns
Data_ssnutritionvspoptotal = [7.10,92.9]
labels = ['Sous nutrition','Population mondiale']
plt.pie(Data_ssnutritionvspoptotal, labels= labels,autopct='%2.2f%%')
```

```
Out[38]: ([<matplotlib.patches.Wedge at 0x1d8e5499910>,
<matplotlib.patches.Wedge at 0x1d8e63bb810>],
[Text(1.0727492911231424, 0.24332890990343878, 'Sous nutrition'),
Text(-1.0727492740365672, -0.24332898523196622, 'Population mondiale')],
[Text(0.5851359769762594, 0.13272485994733021, '7.10%'),
Text(-0.5851359676563093, -0.13272490103561793, '92.90%')])
```

3.2 - Nombre théorique de personne qui pourrait être nourries

```
In [39]: #Combien mange en moyenne un être humain ? Source => Besoin Kcal par jour 24  
#https://nutriandco.com/fr/pages/calcul-apport-calorique-journalier#:~:text=  
Besoin_Kcal_jour_pers = 2400
```

```
In [40]: #On commence par faire une jointure entre le data frame population et Dispo_  
Personne_nourries = pd.merge(Pop_2017,Dispo_alimentaire,on ='Zone' )
```

```
In [41]: #Affichage du nouveau dataframe  
display(Personne_nourries)
```

	Zone	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations (Kca
0	Afghanistan	36296113.0	Abats Comestible	animale	0.0	0.0
1	Afghanistan	36296113.0	Agrumes, Autres	vegetale	0.0	0.0
2	Afghanistan	36296113.0	Aliments pour enfants	vegetale	0.0	0.0
3	Afghanistan	36296113.0	Ananas	vegetale	0.0	0.0
4	Afghanistan	36296113.0	Bananes	vegetale	0.0	0.0
...	...	...	...	...	...	...
15411	Zimbabwe	14236595.0	Viande de Suides	animale	0.0	0.0
15412	Zimbabwe	14236595.0	Viande de Volailles	animale	0.0	0.0
15413	Zimbabwe	14236595.0	Viande, Autre	animale	0.0	1000.0
15414	Zimbabwe	14236595.0	Vin	vegetale	0.0	0.0
15415	Zimbabwe	14236595.0	Épices, Autres	vegetale	0.0	0.0

15416 rows × 7 columns

```
In [42]: #Création de la colonne dispo_kcal avec calcul des kcal disponibles mondiale
dispo_kcal = Personne_nourries.groupby(['Zone']).agg({'Disponibilité alimentaire': lambda x: x.sum()})
display(dispo_kcal)
```

Zone	Disponibilité alimentaire (Kcal/personne/jour)	Population
Afghanistan	2087.0	36296113.0
Afrique du Sud	3020.0	57009756.0
Albanie	3188.0	2884169.0
Algérie	3293.0	41389189.0
Allemagne	3503.0	82658409.0
...	...	...
Émirats arabes unis	3275.0	9487203.0
Équateur	2346.0	16785361.0
États-Unis d'Amérique	3682.0	325084756.0
Éthiopie	2129.0	106399924.0
Îles Salomon	2383.0	636039.0

172 rows × 2 columns

```
In [43]: #Calcul du nombre d'humains pouvant être nourris
dispo_kcal ['Dispo_Kcal_jour'] = dispo_kcal['Disponibilité alimentaire (Kcal
dispo_kcal ['nb_pers_nourries_jour'] = dispo_kcal['Dispo_Kcal_jour']/Besoir
display(dispo_kcal)
Nb_pers_nourries_total_jour = dispo_kcal['nb_pers_nourries_jour'].sum()
print("le nombre de personnes pouvant être nourries au niveau mondial est de
```

	Disponibilité alimentaire (Kcal/personne/jour)	Population	Dispo_Kcal_jour	nb_pers_nourrie
Zone				
Afghanistan	2087.0	36296113.0	7.574999e+10	3.15624
Afrique du Sud	3020.0	57009756.0	1.721695e+11	7.1737
Albanie	3188.0	2884169.0	9.194731e+09	3.8311
Algérie	3293.0	41389189.0	1.362946e+11	5.6789
Allemagne	3503.0	82658409.0	2.895524e+11	1.2064
...	...	...	...	...
Émirats arabes unis	3275.0	9487203.0	3.107059e+10	1.2946
Équateur	2346.0	16785361.0	3.937846e+10	1.6407
États-Unis d'Amérique	3682.0	325084756.0	1.196962e+12	4.9873
Éthiopie	2129.0	106399924.0	2.265254e+11	9.4385
Îles Salomon	2383.0	636039.0	1.515681e+09	6.3153

172 rows × 4 columns

le nombre de personnes pouvant être nourries au niveau mondial est de 8716243594.72125

```
In [44]: #Population totale en 2017
Pop_total_mondial_2017 = 7500000000
Ratio_pop_tot_sur_nourrissble_2017 = (Nb_pers_nourries_total_jour/Pop_total)
print(Ratio_pop_tot_sur_nourrissble_2017 )
```

116.21658126295

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```
In [45]: #Transfert des données avec les végétaux dans un nouveau dataframe
Personne_nourries_veg = Personne_nourries.loc[Personne_nourries['Origine']=='Végétaux']
display(Personne_nourries_veg)
```

	Zone	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	(Kcal/l
1	Afghanistan	36296113.0	Agrumes, Autres	vegetale	0.0	0.0	
2	Afghanistan	36296113.0	Aliments pour enfants	vegetale	0.0	0.0	
3	Afghanistan	36296113.0	Ananas	vegetale	0.0	0.0	
4	Afghanistan	36296113.0	Bananes	vegetale	0.0	0.0	
6	Afghanistan	36296113.0	Bière	vegetale	0.0	0.0	
...	...	...	...	...	...	...	
15406	Zimbabwe	14236595.0	Sésame	vegetale	0.0	0.0	
15407	Zimbabwe	14236595.0	Thé	vegetale	0.0	0.0	
15408	Zimbabwe	14236595.0	Tomates	vegetale	0.0	0.0	
15414	Zimbabwe	14236595.0	Vin	vegetale	0.0	0.0	
15415	Zimbabwe	14236595.0	Épices, Autres	vegetale	0.0	0.0	

11751 rows × 19 columns

```
In [46]: #Calcul du nombre de kcal disponible pour les végétaux
dispo_kcal_veg = Personne_nourries_veg.groupby(['Zone']).agg({'Disponibilité
display(dispo_kcal_veg)
```

Zone	Disponibilité alimentaire (Kcal/personne/jour)	Population
Afghanistan	1871.0	36296113.0
Afrique du Sud	2533.0	57009756.0
Albanie	2203.0	2884169.0
Algérie	2915.0	41389189.0
Allemagne	2461.0	82658409.0
...	...	...
Émirats arabes unis	2718.0	9487203.0
Équateur	1732.0	16785361.0
États-Unis d'Amérique	2698.0	325084756.0
Éthiopie	2005.0	106399924.0
Îles Salomon	2187.0	636039.0

172 rows × 2 columns

```
In [47]: #Calcul du nombre d'humains pouvant être nourris avec les végétaux
dispo_kcal_veg ['Dispo_kcal_jour_veg'] = dispo_kcal_veg['Disponibilité alim
dispo_kcal_veg ['nb_pers_nourries_kcal_veg'] = dispo_kcal_veg['Dispo_kcal_j
display(dispo_kcal_veg)
Nb_pers_nourries_total_jour_veg = dispo_kcal_veg['nb_pers_nourries_kcal_veg'
print("le nombre de personnes pouvant être nourries au niveau mondial en proc
```

	Disponibilité alimentaire (Kcal/personne/jour)	Population	Dispo_kcal_jour_veg	nb_pers_no
Zone				
Afghanistan	1871.0	36296113.0	6.791003e+10	
Afrique du Sud	2533.0	57009756.0	1.444057e+11	
Albanie	2203.0	2884169.0	6.353824e+09	
Algérie	2915.0	41389189.0	1.206495e+11	
Allemagne	2461.0	82658409.0	2.034223e+11	
...	...	...	...	
Émirats arabes unis	2718.0	9487203.0	2.578622e+10	
Équateur	1732.0	16785361.0	2.907225e+10	
États-Unis d'Amérique	2698.0	325084756.0	8.770787e+11	
Éthiopie	2005.0	106399924.0	2.133318e+11	
Îles Salomon	2187.0	636039.0	1.391017e+09	

172 rows × 4 columns

le nombre de personnes pouvant être nourries au niveau mondial en produit végétal est de 7191985088.125416

3.4 - Utilisation de la disponibilité intérieure

```
In [48]: #Calcul de la disponibilité totale
Dispo_inté_tot = Dispo_alimentaire['Disponibilité intérieure'].sum()
print("la disponibilité totale est", Dispo_inté_tot)
```

la disponibilité totale est 9848994000.0

```
In [49]: display(Dispo_alimentaire)
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibi alimenta (Kcal/personne/jc
0	Afghanistan	Abats Comestible	animale	0.0	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	
...	...	...	...	...	...	
15600	Îles Salomon	Viande de Suides	animale	0.0	0.0	4
15601	Îles Salomon	Viande de Volailles	animale	0.0	0.0	1
15602	Îles Salomon	Viande, Autre	animale	0.0	0.0	
15603	Îles Salomon	Vin	vegetale	0.0	0.0	
15604	Îles Salomon	Épices, Autres	vegetale	0.0	0.0	

15605 rows × 18 columns

```
In [50]: #création d'une boucle for pour afficher les différentes valeurs en fonction
Data_dispo_int = ["Aliments pour animaux", "Nourriture", "Pertes", "Semences", "
for Dispo_int in Data_dispo_int:
    print("Proportion pour", Dispo_int, (Dispo_alimentaire[Dispo_int].sum()/Di
```

Proportion pour Aliments pour animaux 13.242418464261426

Proportion pour Nourriture 49.51021393657058

Proportion pour Pertes 4.606541541196999

Proportion pour Semences 1.5705258831511117

Proportion pour Traitement 22.384895350733284

Proportion pour Autres Utilisations 8.782856401374598

```
In [51]: Repartition_dispo_alimentaire = Dispo_alimentaire[['Aliments pour animaux', '
Repartition_dispo_alimentaire['Zone'] = 'All'
Repartition_dispo_groupe = Repartition_dispo_alimentaire.groupby(['Zone']).a
Repartition_dispo_groupe = Repartition_dispo_groupe.reset_index()
df_melt = pd.melt(Repartition_dispo_groupe, id_vars=['Zone'], var_name='repart
display(df_melt)
```


C:\Users\726628\AppData\Local\Temp\1\ipykernel_29368\2292461064.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

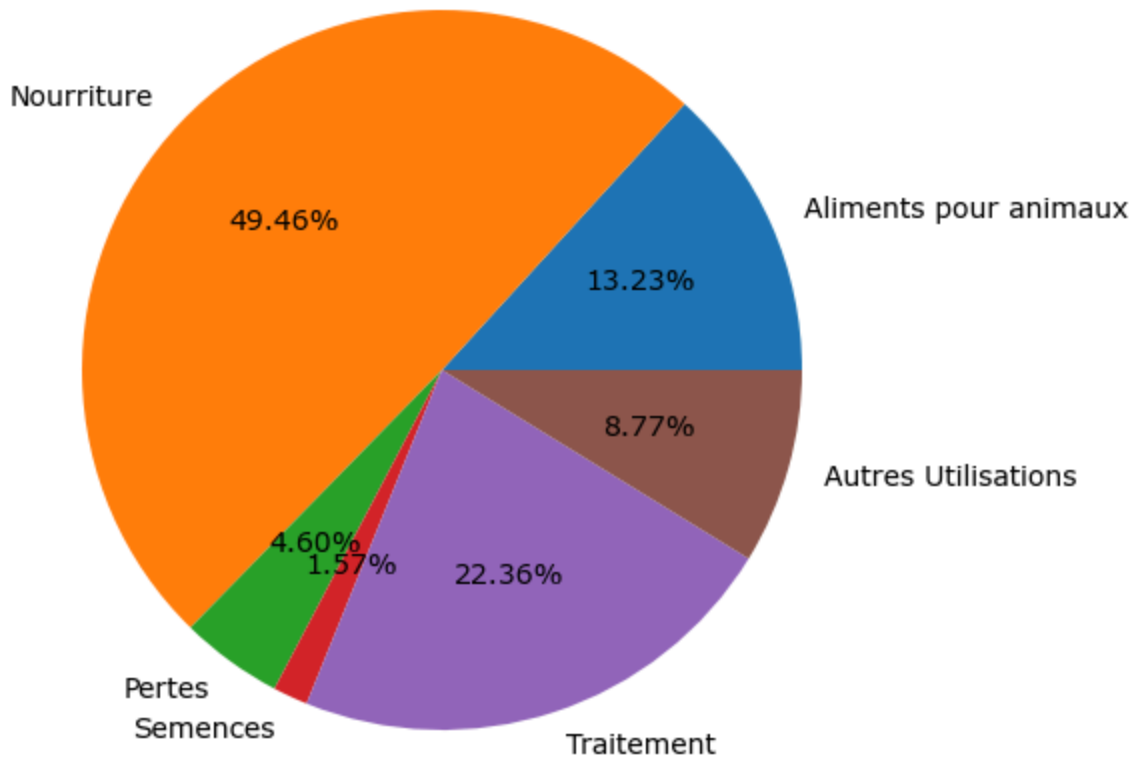
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

Repartition_dispo_alimentaire['Zone'] = 'All'

	Zone	repartition	valeurs
0	All	Aliments pour animaux	1.304245e+09
1	All	Nourriture	4.876258e+09
2	All	Pertes	4.536980e+08
3	All	Semences	1.546810e+08
4	All	Traitement	2.204687e+09
5	All	Autres Utilisations	8.650230e+08

```
In [52]: import matplotlib.pyplot as plt
plt.pie(x=df_melt['valeurs'], labels=df_melt['repartition'], autopct='%2.2f%%')
```

```
Out[52]: ([<matplotlib.patches.Wedge at 0x1d8e63d1910>,
<matplotlib.patches.Wedge at 0x1d8e6422fd0>,
<matplotlib.patches.Wedge at 0x1d8e6430790>,
<matplotlib.patches.Wedge at 0x1d8e6423fd0>,
<matplotlib.patches.Wedge at 0x1d8e6433550>,
<matplotlib.patches.Wedge at 0x1d8e643cf10>],
[Text(1.0063537450463638, 0.4441307688408431, 'Aliments pour animaux'),
Text(-0.7999957113477486, 0.754987987868158, 'Nourriture'),
Text(-0.646979358138829, -0.8896166085130544, 'Pertes'),
Text(-0.4634678736291093, -0.9975958751487057, 'Semences'),
Text(0.3428190463756664, -1.0452153373549773, 'Traitement'),
Text(1.0584724828480327, -0.2993927237818598, 'Autres Utilisations')],
[Text(0.5489202245707439, 0.24225314664045985, '13.23%'),
Text(-0.43636129709877197, 0.41181162974626795, '49.46%'),
Text(-0.35289783171208855, -0.4852454228253023, '4.60%'),
Text(-0.25280065834315046, -0.5441432046265667, '1.57%'),
Text(0.18699220711399986, -0.5701174567390784, '22.36%'),
Text(0.5773486270080178, -0.1633051220628326, '8.77%')])
```



3.5 - Utilisation des céréales

```
In [53]: #Création d'une liste avec toutes les variables
liste_céréales = ["Blé", "Riz (Eq Blanchi)", "Orge", "Maïs", "Seigle", "Avoine", "

In [54]: #Création d'un dataframe avec les informations uniquement pour ces céréales
Personne_nourries_céréales = Dispo_alimentaire.loc[Dispo_alimentaire['Produit']
display(Personne_nourries_céréales)
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)
7	Afghanistan	Blé	vegetale	0.0	0.0	1369.0
32	Afghanistan	Maïs	vegetale	200000.0	0.0	21.0
34	Afghanistan	Millet	vegetale	0.0	0.0	3.0
40	Afghanistan	Orge	vegetale	360000.0	0.0	26.0
47	Afghanistan	Riz (Eq Blanchi)	vegetale	0.0	0.0	141.0
...	...	...	...	...	...	...
15537	Îles Salomon	Blé	vegetale	0.0	0.0	184.0
15568	Îles Salomon	Maïs	vegetale	0.0	0.0	1.0
15575	Îles Salomon	Orge	vegetale	0.0	0.0	0.0
15591	Îles Salomon	Riz (Eq Blanchi)	vegetale	0.0	12000.0	623.0
15593	Îles Salomon	Sorgho	vegetale	0.0	0.0	0.0

1325 rows × 18 columns

```
In [55]: #Affichage de la proportion d'alimentation animale
Dispo_int_cereale = Personne_nourries_céréales['Disponibilité intérieure'].sum()
Datafram_ble = Personne_nourries_céréales.loc[Personne_nourries_céréales['Pr
Dispo_total_int_blé = Datafram_ble['Disponibilité intérieure'].sum()
Proportion_dispo_int_blé = (Dispo_total_int_blé/Dispo_int_cereale)*100
print(Proportion_dispo_int_blé)
```

28.556167351820584

```
In [56]: Utilisation_pour_animaux = Personne_nourries_céréales['Aliments pour animaux
Utilisation_pour_humains = Personne_nourries_céréales['Nourriture'].sum()
Proportion_utilisation_pour_animaux = round((Utilisation_pour_animaux/Dispo
print("la proportion d'alimentation animale",Proportion_utilisation_pour_an
Proportion_utilisation_pour_humains = round((Utilisation_pour_humains/Dispo
print("la proportion d'alimentation humains",Proportion_utilisation_pour_hum
```

la proportion d'alimentation animale 35.91

la proportion d'alimentation humains 43.02

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

```
In [57]: #Création de la colonne proportion par pays
Data_nutrition_2017['proportion_pays'] = round(((Data_nutrition_2017['sous_nutrition']/Data_nutrition_2017['Population'])*100))

In [58]: #affichage après tri des 10 pires pays
Liste_10_pays_SS_alimenté_2017 = Data_nutrition_2017.sort_values(['proportion_pays'], ascending=True)
display(Liste_10_pays_SS_alimenté_2017)
```

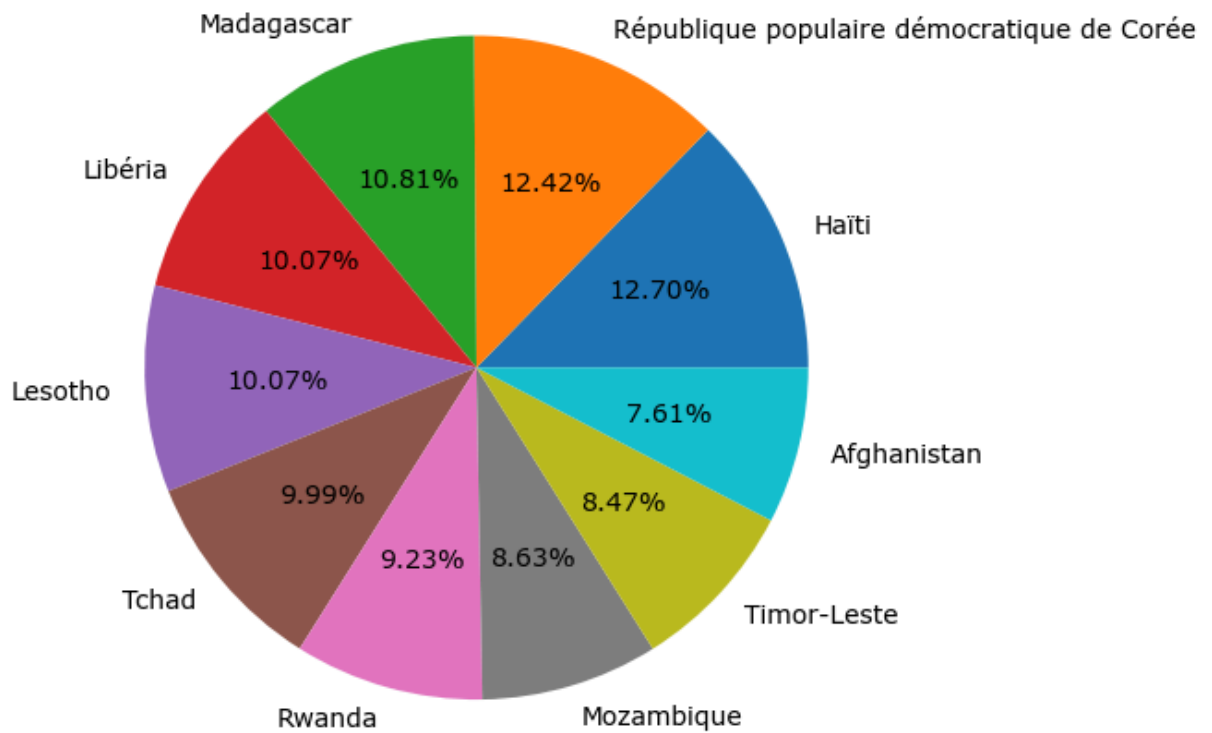
	Zone	Population	sous_nutrition	proportion_pays
78	Haïti	10982366.0	5300000.0	48.26
157	République populaire démocratique de Corée	25429825.0	12000000.0	47.19
108	Madagascar	25570512.0	10500000.0	41.06
103	Libéria	4702226.0	1800000.0	38.28
100	Lesotho	2091534.0	800000.0	38.25
183	Tchad	15016753.0	5700000.0	37.96
161	Rwanda	11980961.0	4200000.0	35.06
121	Mozambique	28649018.0	9400000.0	32.81
186	Timor-Leste	1243258.0	400000.0	32.17
0	Afghanistan	36296113.0	10500000.0	28.93

```
In [59]: #Graphique des 10 pays ou la proportion de personnes en sous nutrition est la plus élevée
import matplotlib.pyplot as plt
plt.pie(x=Liste_10_pays_SS_alimenté_2017['proportion_pays'], labels=Liste_10_pays_SS_alimenté_2017['Zone'])
```

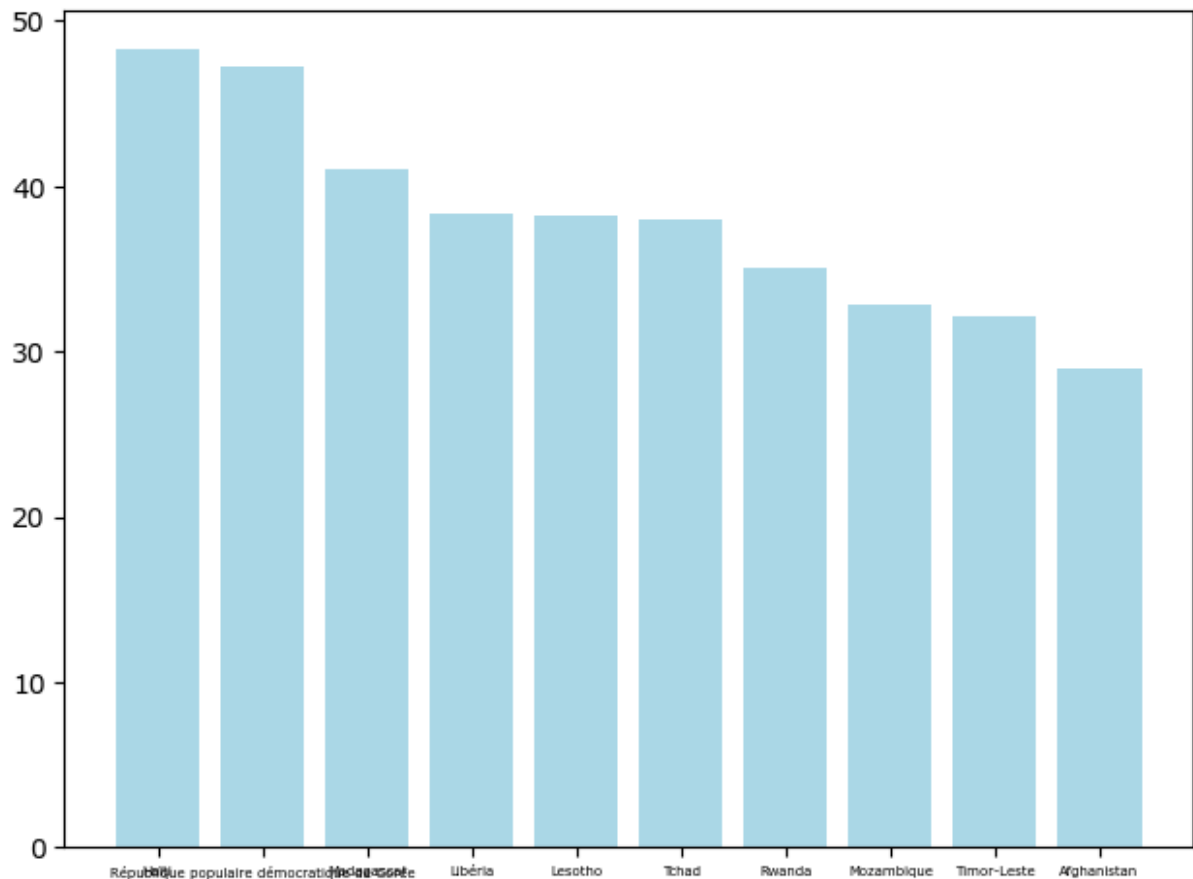
```

Out[59]: ([<matplotlib.patches.Wedge at 0x1d8e676e690>,
<matplotlib.patches.Wedge at 0x1d8e67af6d0>,
<matplotlib.patches.Wedge at 0x1d8e67bcd90>,
<matplotlib.patches.Wedge at 0x1d8e67bc5d0>,
<matplotlib.patches.Wedge at 0x1d8e67bf790>,
<matplotlib.patches.Wedge at 0x1d8e67d1150>,
<matplotlib.patches.Wedge at 0x1d8e67d2710>,
<matplotlib.patches.Wedge at 0x1d8e67d3ed0>,
<matplotlib.patches.Wedge at 0x1d8e67e1510>,
<matplotlib.patches.Wedge at 0x1d8e67d0f50>],
[Text(1.0135890557918668, 0.4273607679454821, 'Haïti'),
Text(0.41066889098990156, 1.020466100354698, 'République populaire démocr
atique de Corée'),
Text(-0.37413712870468835, 1.0344183916214036, 'Madagascar'),
Text(-0.9274162943189939, 0.5915226259676174, 'Libéria'),
Text(-1.0976780210951154, -0.07143501945622713, 'Lesotho'),
Text(-0.8447948741295602, -0.7045009727774836, 'Tchad'),
Text(-0.2954984782356084, -1.0595662552952694, 'Rwanda'),
Text(0.31367639468256103, -1.054327804536592, 'Mozambique'),
Text(0.809068019606483, -0.745257633070634, 'Timor-Leste'),
Text(1.068682474556796, -0.26061037695219125, 'Afghanistan')],
[Text(0.5528667577046545, 0.2331058734248084, '12.70%'),
Text(0.224001213267219, 0.5566178729207443, '12.42%'),
Text(-0.20407479747528454, 0.5642282136116746, '10.81%'),
Text(-0.5058634332649057, 0.3226487050732458, '10.07%'),
Text(-0.598733466051881, -0.038964556067032974, '10.07%'),
Text(-0.46079720407066915, -0.3842732578786274, '9.99%'),
Text(-0.16118098812851364, -0.5779452301610559, '9.23%'),
Text(0.17109621528139693, -0.5750878933835956, '8.63%'),
Text(0.4413098288762634, -0.406504163493073, '8.47%'),
Text(0.582917713394616, -0.1421511147011952, '7.61%')])

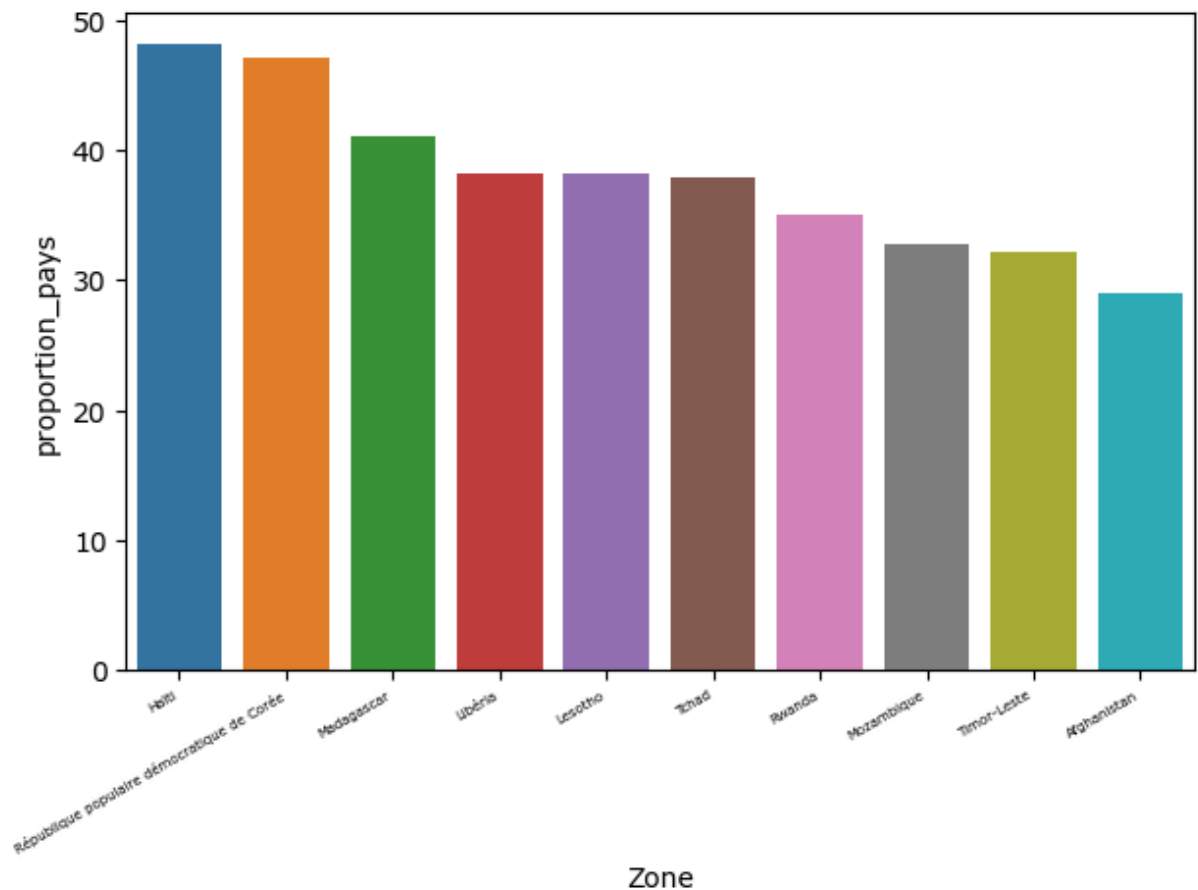
```



```
In [60]: import matplotlib.pyplot as plt
plt.title = ('10 pays ou la proposition de personnes en sous nutrition est l
plt.rcParams.update({'xtick.labelsize':5,})
plt.xlabel= ('Pays')
plt.ylabel= ('la proposition de personnes en sous nutrition')
plt.bar(height = Liste_10_pays_SS_alimenté_2017['proportion_pays'],x=Liste_1
plt.show()
```



```
In [61]: import seaborn as sns
import matplotlib.pyplot as plt
sns.barplot(data = Liste_10_pays_SS_alimenté_2017,x='Zone',y='proportion_pay
plt.rcParams.update({'xtick.labelsize':10,})
plt.xticks(rotation=30,ha='right')
plt.title = ('10 pays ou la proposition de personnes en sous nutrition est l
plt.xlabel= ('Pays')
plt.ylabel= ('la proposition de personnes en sous nutrition')
plt.show()
print(plt.rcParams['figure.figsize'])
print(plt.rcParams['figure.titlesize'])
```



[6.4, 4.8]
large

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

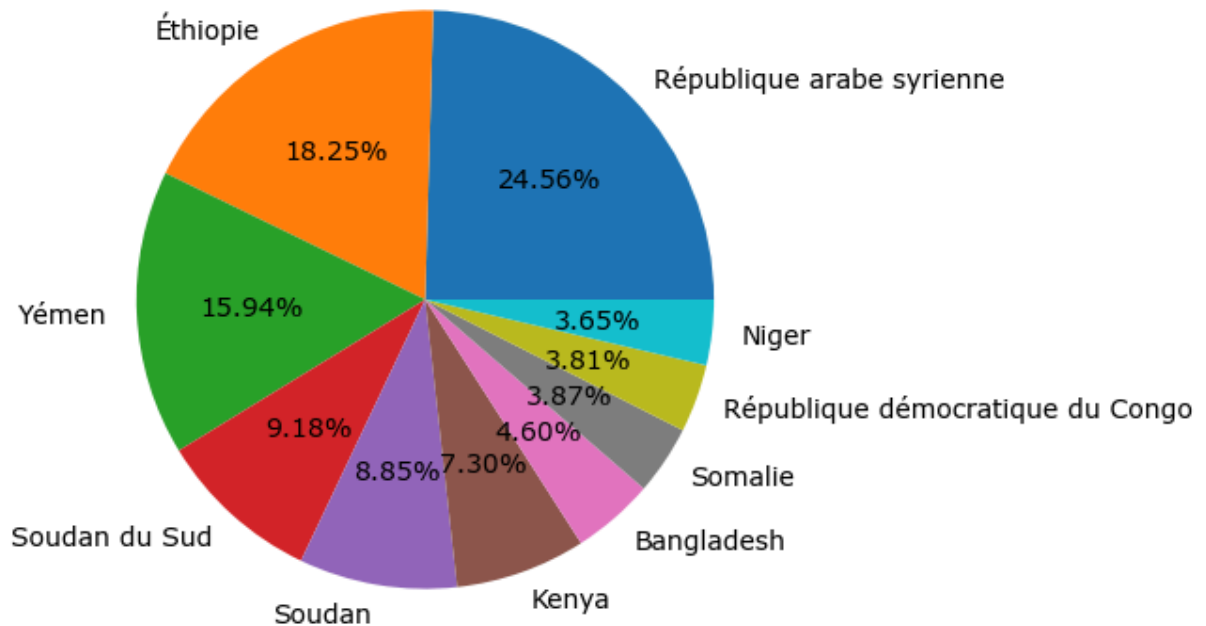
```
In [62]: #calcul du total de l'aide alimentaire par pays
Total_aide_aliment_from2013 = Aide_alimentaire.groupby(['Zone']).agg({'Valeu

In [63]: #affichage après trie des 10 pays qui ont bénéficié le plus de l'aide alimen
Liste_10pays_le_plus_aidé_since_2013 = Total_aide_aliment_from2013.sort_valu
Liste_10pays_le_plus_aidé_since_2013 = Liste_10pays_le_plus_aidé_since_2013.
display(Liste_10pays_le_plus_aidé_since_2013)
```


	Zone	Valeur
0	République arabe syrienne	1858943000
1	Éthiopie	1381294000
2	Yémen	1206484000
3	Soudan du Sud	695248000
4	Soudan	669784000
5	Kenya	552836000
6	Bangladesh	348188000
7	Somalie	292678000
8	République démocratique du Congo	288502000
9	Niger	276344000

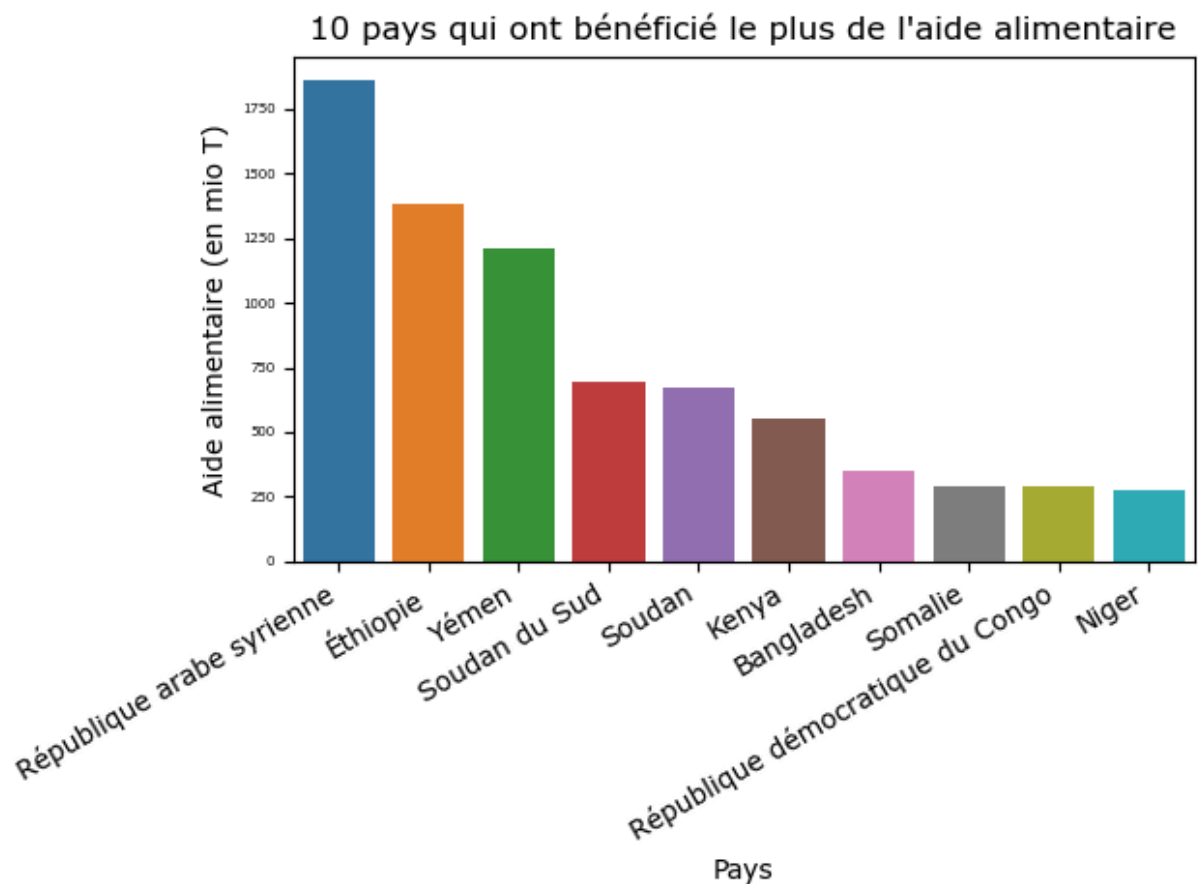
```
In [64]: import matplotlib.pyplot as plt
plt.pie(x=Liste_10pays_le_plus_aidé_since_2013['Valeur'], labels=Liste_10pays
```

```
Out[64]: ([<matplotlib.patches.Wedge at 0x1d8e552a710>,
<matplotlib.patches.Wedge at 0x1d8e681db50>,
<matplotlib.patches.Wedge at 0x1d8e680b250>,
<matplotlib.patches.Wedge at 0x1d8e63c3490>,
<matplotlib.patches.Wedge at 0x1d8e6808e90>,
<matplotlib.patches.Wedge at 0x1d8e67ff5d0>,
<matplotlib.patches.Wedge at 0x1d8e676f290>,
<matplotlib.patches.Wedge at 0x1d8e549d690>,
<matplotlib.patches.Wedge at 0x1d8e549e310>,
<matplotlib.patches.Wedge at 0x1d8e67ffed0>],
[Text(0.7885973740867571, 0.7668860290704685, 'République arabe syrienn
e'),
Text(-0.5705495273113964, 0.9404643730013074, 'Éthiopie'),
Text(-1.0987112392148815, -0.05323169002482884, 'Yémen'),
Text(-0.7361651050222733, -0.817349948398815, 'Soudan du Sud'),
Text(-0.18253643500069677, -1.0847490262255304, 'Soudan'),
Text(0.36752078804420035, -1.03678757243486, 'Kenya'),
Text(0.7208267182760452, -0.830908443945172, 'Bangladesh'),
Text(0.9138706166322357, -0.6122421874195029, 'Somalie'),
Text(1.033654779569953, -0.3762416732263874, 'République démocratique du
Congo'),
Text(1.092774627425654, -0.12587141714751257, 'Niger')],
[Text(0.4301440222291402, 0.4183014704020737, '24.56%'),
Text(-0.3112088330789434, 0.5129805670916222, '18.25%'),
Text(-0.5992970395717535, -0.029035467286270274, '15.94%'),
Text(-0.4015446027394217, -0.4458272445811718, '9.18%'),
Text(-0.09956532818219822, -0.5916812870321074, '8.85%'),
Text(0.20046588438774562, -0.5655204940553781, '7.30%'),
Text(0.3931782099687519, -0.4532227876064574, '4.60%'),
Text(0.49847488179940125, -0.3339502840470015, '3.87%'),
Text(0.5638116979472471, -0.20522273085075673, '3.81%'),
Text(0.5960588876867203, -0.06865713662591594, '3.65%')])]
```



```
In [148... import seaborn as sns
import matplotlib.pyplot as plt
g=sns.barplot(data = Liste_10pays_le_plus_aidé_since_2013,x='Zone',y='Valeur')
g.set_title("10 pays qui ont bénéficié le plus de l'aide alimentaire")
g.set_yticklabels([0,250,500,750,1000,1250,1500,1750])
g.set_ylabel("Aide alimentaire (en mio T)")
g.set_xlabel("Pays")
plt.rcParams.update({'xtick.labelsize':10,'ytick.labelsize':5})
plt.xticks(rotation=30,ha='right')
plt.ylabel= ("Aide alimentaire")
plt.show()
```

C:\Users\726628\AppData\Local\Temp\1\ipykernel_29368\2000790552.py:5: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
g.set_yticklabels([0,250,500,750,1000,1250,1500,1750])



3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
In [68]: #Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis gr
Data_2013_2016_Aide_alimentaire = Aide_alimentaire[['Zone', 'Année', 'Valeur']]
Data_regroupé_enzonéetannée = Data_2013_2016_Aide_alimentaire.groupby(['Zone
Data_regroupé_enzonéetannée = Data_regroupé_enzonéetannée.reset_index()
display(Data_regroupé_enzonéetannée)
```

	Zone	Année	Valeur
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000
...	...	...	...
223	Égypte	2013	1122000
224	Équateur	2013	1362000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

228 rows × 3 columns

```
In [69]: #Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de l'aide
Data_top5_aide = Data_regroupé_enzoneetannée.pivot_table(index='Zone',values='Valeur')
Data_top5_aide = Data_top5_aide.reset_index()
display(Data_top5_aide)

Liste_pays_top5 = Data_top5_aide['Zone'].to_list()
display(Liste_pays_top5)
```

	Zone	Valeur
0	République arabe syrienne	1858943000
1	Éthiopie	1381294000
2	Yémen	1206484000
3	Soudan du Sud	695248000
4	Soudan	669784000

```
['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan']
```

```
In [70]: #On filtre sur le dataframe avec notre liste
Liste_5_pays_dataframe = Data_regroupé_enzoneetannée.loc[Liste_pays_top5]
display(Liste_5_pays_dataframe)
Liste_5_pays_dataframe.dtypes
Liste_5_pays_dataframe['Année'] = Liste_5_pays_dataframe['Année'].astype(object)
Liste_5_pays_dataframe.dtypes
```

	Zone	Année	Valeur
157	République arabe syrienne	2013	563566000
158	République arabe syrienne	2014	651870000
159	République arabe syrienne	2015	524949000
160	République arabe syrienne	2016	118558000
189	Soudan	2013	330230000
190	Soudan	2014	321904000
191	Soudan	2015	17650000
192	Soudan du Sud	2013	196330000
193	Soudan du Sud	2014	450610000
194	Soudan du Sud	2015	48308000
214	Yémen	2013	264764000
215	Yémen	2014	103840000
216	Yémen	2015	372306000
217	Yémen	2016	465574000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

C:\Users\726628\AppData\Local\Temp\1\ipykernel_29368\1779708694.py:5: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

Liste_5_pays_dataframe['Année']=Liste_5_pays_dataframe['Année'].astype(object)

Out[70]: Zone object
Année object
Valeur int64
dtype: object

```
In [109... import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
plt.figure(figsize=(12,5))
ax=sns.lineplot(Liste_5_pays_dataframe, x='Année', y='Valeur',errorbar=None,
ax.set_title("Variation des 5 pays qui ont le plus bénéficiées de l'aide ali
ax.set_xlabel("Année")
ax.set_ylabel("Aide alimentaire en Mio kg")
ax.set_xticklabels([0,2013,"",2014,"",2015,"",2016])
ax.set_yticklabels([0,100,200,300,400,500,600,700])
```

```
plt.legend(bbox_to_anchor=(1,1))
plt.show()
```

C:\Users\726628\AppData\Local\anaconda3\Nouveau dossier\Lib\site-packages\seaborn_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

```
with pd.option_context('mode.use_inf_as_na', True):
```

C:\Users\726628\AppData\Local\anaconda3\Nouveau dossier\Lib\site-packages\seaborn_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

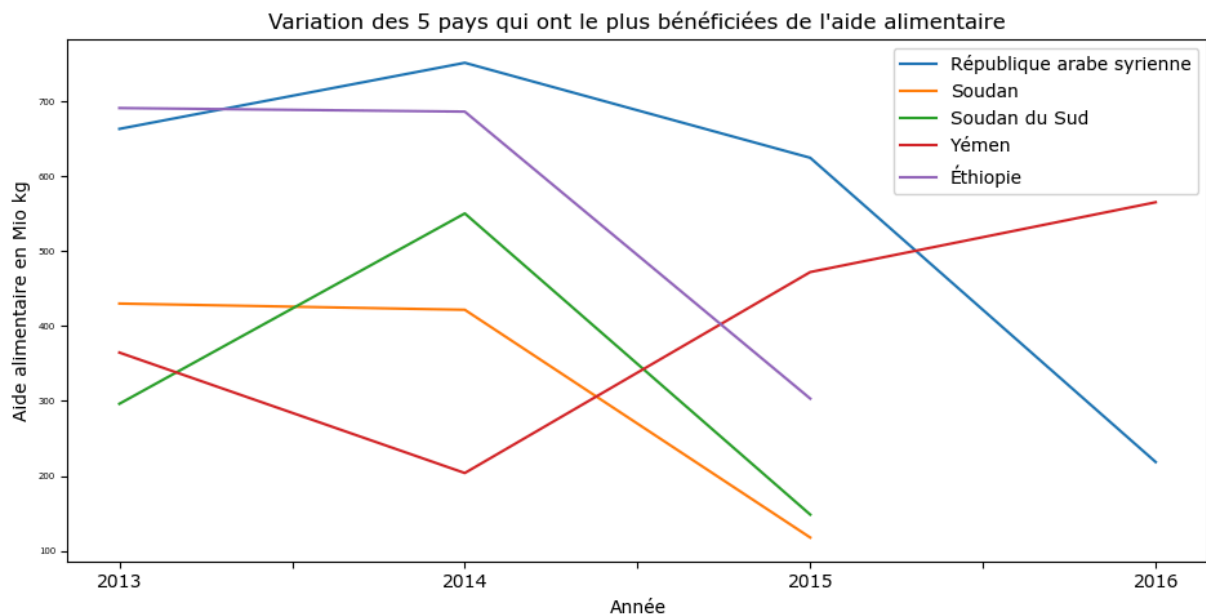
```
with pd.option_context('mode.use_inf_as_na', True):
```

C:\Users\726628\AppData\Local\Temp\1\ipykernel_29368\91264608.py:9: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.

```
ax.set_xticklabels([0,2013,"",2014,"",2015,"",2016])
```

C:\Users\726628\AppData\Local\Temp\1\ipykernel_29368\91264608.py:10: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.

```
ax.set_yticklabels([0,100,200,300,400,500,600,700])
```



```
In [ ]: # Affichage des pays avec l'aide alimentaire par année
Data_regroupe_ensembleetannée = Data_2013_2016_Aide_alimentaire.groupby(['Zone
display(Data_regroupe_ensembleetannée)
```

3.9 - Pays avec le moins de disponibilité par habitant

```
In [159... #Calcul de la disponibilité en kcal par personne par jour par pays
dispo_kcal_hab = Personne_nourries.groupby(['Zone']).agg({'Disponibilité ali
display(dispo_kcal_hab)
dispo_kcal_hab['Proportion_dispo_Kcal_habitant'] = round((dispo_kcal_hab/Besc
display(dispo_kcal_hab)
```

Disponibilité alimentaire (Kcal/personne/jour)	
Zone	
Afghanistan	2087.0
Afrique du Sud	3020.0
Albanie	3188.0
Algérie	3293.0
Allemagne	3503.0
...	...
Émirats arabes unis	3275.0
Équateur	2346.0
États-Unis d'Amérique	3682.0
Éthiopie	2129.0
Îles Salomon	2383.0

172 rows × 1 columns

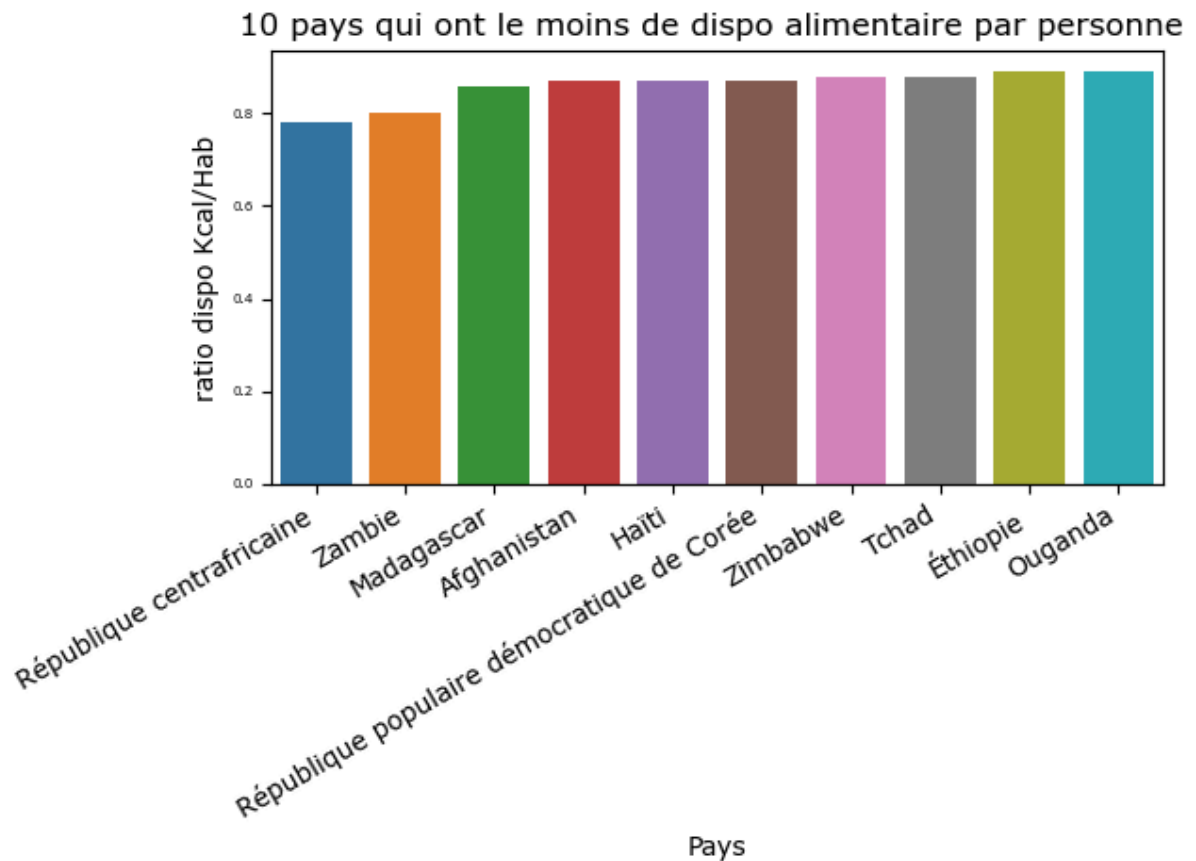
	Disponibilité alimentaire (Kcal/personne/jour)	Proportion_dispo_Kcal_habitant
Zone		
Afghanistan	2087.0	0.87
Afrique du Sud	3020.0	1.26
Albanie	3188.0	1.33
Algérie	3293.0	1.37
Allemagne	3503.0	1.46
...	...	...
Émirats arabes unis	3275.0	1.36
Équateur	2346.0	0.98
États-Unis d'Amérique	3682.0	1.53
Éthiopie	2129.0	0.89
Îles Salomon	2383.0	0.99

172 rows × 2 columns

In [161... *#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne*
 Pays_dispo_alim_lesplusfaibles_par_pers = dispo_kcal_hab.sort_values(['Proportion_dispo_Kcal_habitant'])
 Pays_dispo_alim_lesplusfaibles_par_pers = Pays_dispo_alim_lesplusfaibles_par_pers.sort_values(['Disponibilité alimentaire (Kcal/personne/jour)'])
 Pays_dispo_alim_lesplusfaibles_par_pers

```
h=sns.barplot(data = Pays_dispo_alim_lesplusfaibles_par_pers,x='Zone',y='Pro
h.set_title("10 pays qui ont le moins de dispo alimentaire par personne ")
h.set_xlabel("Pays")
h.set_ylabel("ratio dispo Kcal/Hab")
plt.rcParams.update({'xtick.labelsize':10,'ytick.labelsize':5})
plt.xticks(rotation=30,ha='right')
plt.show()
```

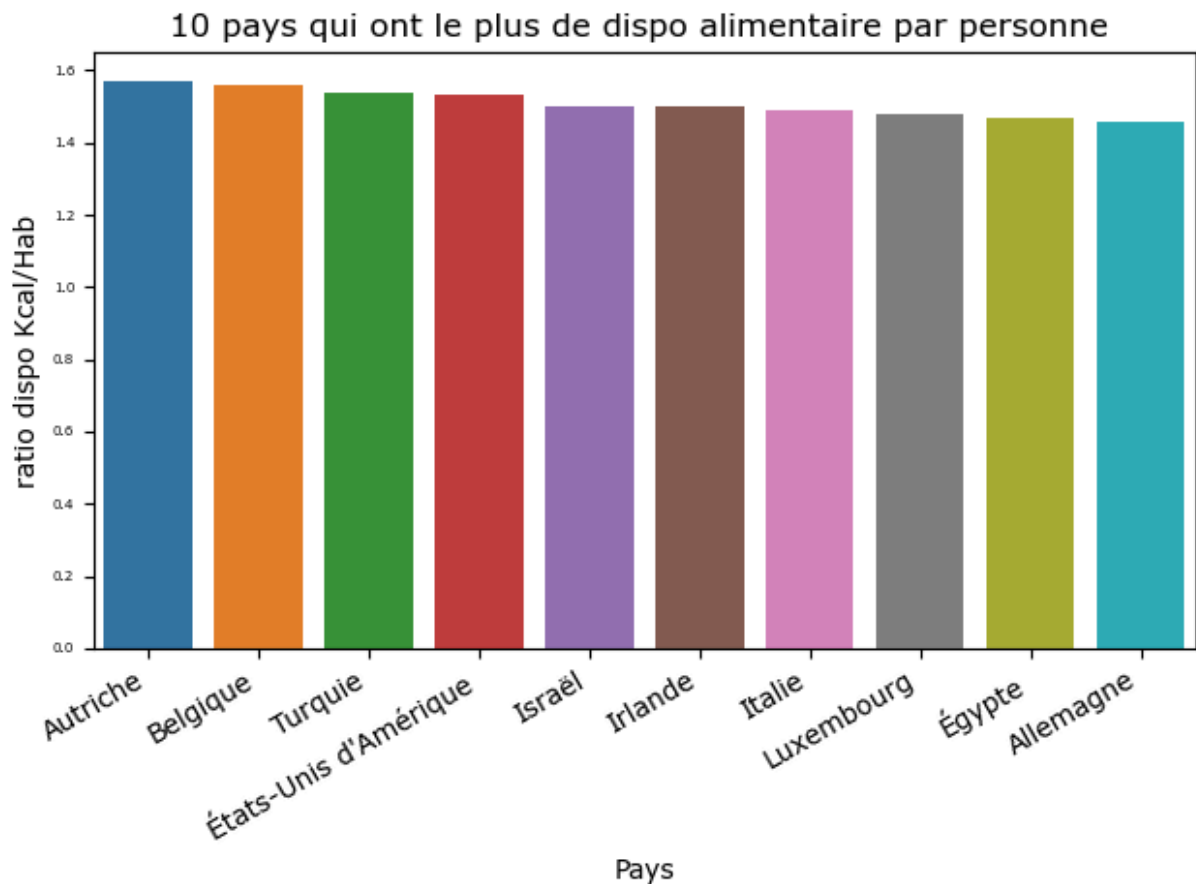
	Zone	Disponibilité alimentaire (Kcal/personne/jour)	Proportion_dispo_Kcal_habitant
0	République centrafricaine	1879.0	0.78
1	Zambie	1924.0	0.80
2	Madagascar	2056.0	0.86
3	Afghanistan	2087.0	0.87
4	Haïti	2089.0	0.87
5	République populaire démocratique de Corée	2093.0	0.87
6	Zimbabwe	2113.0	0.88
7	Tchad	2109.0	0.88
8	Éthiopie	2129.0	0.89
9	Ouganda	2126.0	0.89



3.10 - Pays avec le plus de disponibilité par habitant

```
In [157... #Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
Pays_dispo_alim_lesplusfort = dispo_kcal_hab.sort_values(['Disponibilité ali
Pays_dispo_alim_lesplusfort = Pays_dispo_alim_lesplusfort.reset_index()
display(Pays_dispo_alim_lesplusfort)
h=sns.barplot(data = Pays_dispo_alim_lesplusfort,x='Zone',y='Proportion_disp
h.set_title("10 pays qui ont le plus de dispo alimentaire par personne ")
h.set_xlabel("Pays")
h.set_ylabel("ratio dispo Kcal/Hab")
plt.rcParams.update({'xtick.labelsize':10,'ytick.labelsize':5})
plt.xticks(rotation=30,ha='right')
plt.show()
```

	Zone	Disponibilité alimentaire (Kcal/personne/jour)	Proportion_dispo_Kcal_habitant
0	Autriche	3770.0	1.57
1	Belgique	3737.0	1.56
2	Turquie	3708.0	1.54
3	États-Unis d'Amérique	3682.0	1.53
4	Israël	3610.0	1.50
5	Irlande	3602.0	1.50
6	Italie	3578.0	1.49
7	Luxembourg	3540.0	1.48
8	Égypte	3518.0	1.47
9	Allemagne	3503.0	1.46



3.11 - Exemple de la Thaïlande pour le Manioc

```
In [123... #création d'un dataframe avec uniquement la Thaïlande
Data_Thaïlande = Personne_nourries.loc[Personne_nourries['Zone']=='Thaïlande]
```

```
display(Data_Thaïlande)
```

	Zone	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations (Kcal/
14116	Thaïlande	69209810.0	Abats Comestible	animale	0.0	0.0
14117	Thaïlande	69209810.0	Agrumes, Autres	vegetale	0.0	0.0
14118	Thaïlande	69209810.0	Alcool, non Comestible	vegetale	0.0	358000.0
14119	Thaïlande	69209810.0	Aliments pour enfants	vegetale	0.0	0.0
14120	Thaïlande	69209810.0	Ananas	vegetale	0.0	0.0
...	...	...	...	...	...	...
14206	Thaïlande	69209810.0	Viande de Suides	animale	0.0	0.0
14207	Thaïlande	69209810.0	Viande de Volailles	animale	0.0	0.0
14208	Thaïlande	69209810.0	Viande, Autre	animale	0.0	0.0
14209	Thaïlande	69209810.0	Vin	vegetale	0.0	0.0
14210	Thaïlande	69209810.0	Épices, Autres	vegetale	0.0	0.0

95 rows × 19 columns

```
In [166... #Calcul de la sous nutrition en Thaïlande
Data_sous_nutrition_Thaïlande = Data_nutrition_2017.loc[Data_nutrition_2017[
Data_sous_nutrition_Thaïlande = Data_sous_nutrition_Thaïlande.reset_index()
print(Data_sous_nutrition_Thaïlande)
Part_ss_nutr_sur_pop_thai = round(((Data_sous_nutrition_Thaïlande)['sous_nut
print("Le proportion de sous nutrition en thaïlande est de", Part_ss_nutr_su

    index      Zone  Population  sous_nutrition
0    185  Thaïlande  69209810.0      6200000.0
Le proportion de sous nutrition en thaïlande est de 0      8.96
dtype: float64
```

```
In [245... # On calcule la proportion exportée en fonction de la proportion produite
Manioc_Thai = Data_Thaïlande.loc[Data_Thaïlande['Produit']=='Manioc']
display(Manioc_Thai)
Ratio_prod_export_manioc = round(((Manioc_Thai)['Exportations - Quantité']/(
print("le ration export/prod est",Ratio_prod_export_manioc)
```

	Zone	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	(Kcal/pe
14166	Thaïlande	69209810.0	Manioc	vegetale	1800000.0	2081000.0	

le ration export/prod est 14166 83.41
dtype: float64

```
In [137... # Quelle est la disponibilité par habitant pour la Thaïlande ? Disponibilité
Dispo_Thaïlande_hab = Data_Thaïlande.agg({'Disponibilité alimentaire (Kcal/p
print("la disponibilité intérieur en Thaïlande est de",Dispo_Thaïlande_hab)
```

la disponibilité intérieur en Thaïlande est de Disponibilité alimentaire (Kcal/personne/jour) 2785.0
dtype: float64

Etape 6 - Analyse complémentaires

```
In [272... #Rajouter en dessous toutes les analyses complémentaires suite à la demande de
#"et toutes les infos que tu trouverais utiles pour mettre en relief les pay
#le plus en difficulté au niveau alimentaire"
```

In []: